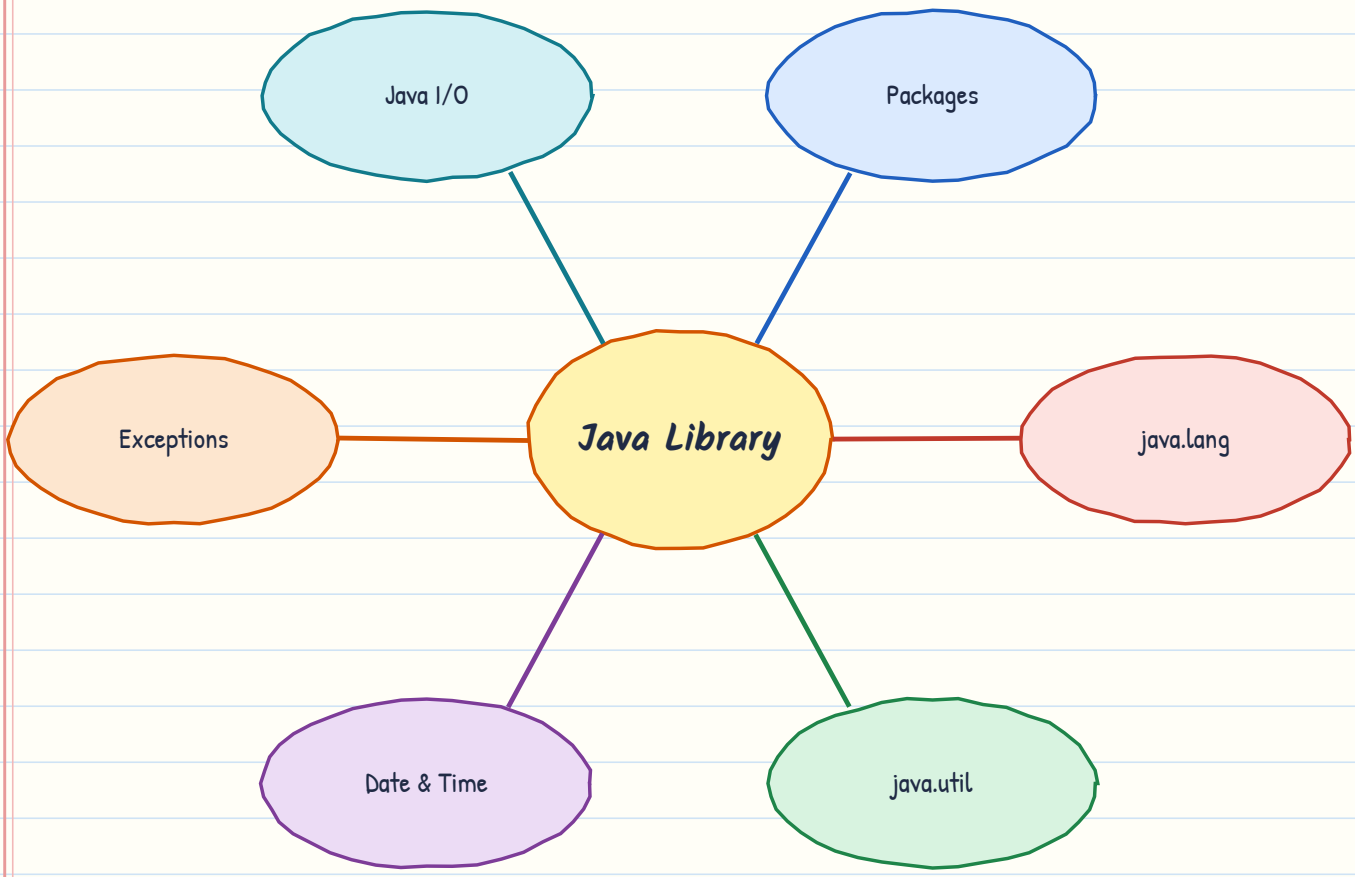
UNIT IV

Packages & Java Library, Exceptions, I/O

Packages | java.lang | java.util | java.time | try-catch | Streams



Mind map: everything you will learn in this unit

How to use these notes

Tip: read the diagrams first, then the text. Try writing each example yourself - code you type sticks in memory, code you only read does not.

1. Packages

A **package** is a folder-like container that groups related classes and interfaces. It avoids name clashes, controls access and keeps big projects organised.

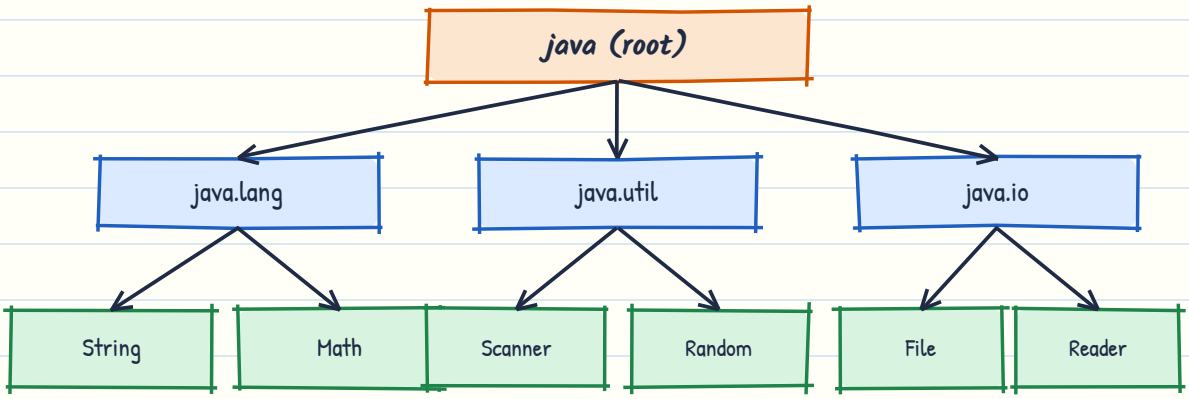


Fig 1: Package = folder -> classes inside

Defining & importing packages

mypack/Msg.java

```
package mypack;                  // MUST be the first line
public class Msg {
    public void show() { System.out.println("Hello from package"); }
}
```

Test.java

```
import mypack.Msg;               // one class
// import mypack.*;               // all classes of package
// import static java.lang.Math.*;   // static members
class Test { public static void main(String[] a) { new Msg().show(); } }
```

```
javac -d . Msg.java       // -d . creates folder mypack/
javac Test.java
java Test
```

- **PATH** = where the OS finds `javac`/`java` tools.
- **CLASSPATH** = where the JVM finds your `.class` files / packages (`-cp` option).
- `java.lang` is imported automatically; all others need `import`.

Access	Same pkg	Other pkg
public	yes	yes
protected	yes	only subclass
default	yes	no
private	no (class only)	no

Packages in Java SE

Package	Contains
java.lang	core: String, Math, Object, wrappers, Thread
java.util	collections, Scanner, Random, Date
java.io	input / output streams, File
java.time	modern date & time API
java.sql	JDBC database classes
javafx.*	GUI classes

2. java.lang Package

- ***Object*** - root class (toString, equals, hashCode, getClass).
- ***String / StringBuffer***, ***Math***, ***Wrapper classes***, ***System***, ***Thread***, ***Exception classes***.

class Math

Method	Result	Method	Result
abs(-5)	5	pow(2,3)	8.0
sqrt(25)	5.0	max(4,9)	9
ceil(4.2)	5.0	floor(4.8)	4.0
round(4.5)	5	random()	0.0 to <1.0

Enumeration (enum)

An ***enum*** is a fixed set of named constants - safer than plain ints.

```
enum Day { MON, TUE, WED }
public class E {
    public static void main(String[] a) {
        Day d = Day.TUE;
        switch (d) { case TUE: System.out.println("Tuesday"); break; default: break; }
        for (Day x : Day.values()) System.out.print(x + " ");   // all constants
        System.out.println(d.ordinal());   // position = 1
    }
}
```

```
> OUTPUT
Tuesday
MON TUE WED 1
```

Wrapper classes, autoboxing & unboxing

A **wrapper class** turns a primitive into an object (needed for collections).

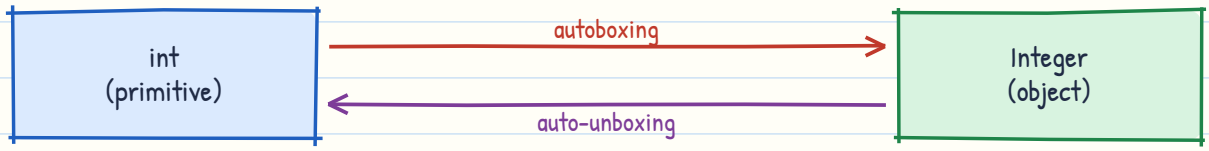


Fig 2: Boxing and unboxing

Primitive	Wrapper	Primitive	Wrapper
byte	Byte	long	Long
short	Short	float	Float
int	Integer	double	Double
char	Character	boolean	Boolean

```
Integer a = 10;                  // autoboxing (int -> Integer)
int b = a;                       // auto-unboxing
int n = Integer.parseInt("123"); // String -> int
String s = Integer.toString(45); // int -> String
System.out.println(Integer.MAX_VALUE);   // 2147483647
```

3. java.util Classes & Interfaces

Class	Use
Scanner	read input
Random	random numbers
Formatter	formatted output (like printf)
ArrayList, HashMap, ...	collections framework

```java
Random r = new Random();
int x = r.nextInt(100);          // 0 to 99
double y = r.nextDouble();       // 0.0 to 1.0
Formatter f = new Formatter();
f.format("%s scored %d", "Ravi", 90);
System.out.println(f);           // Ravi scored 90
```

Time package (java.time)

Class	Represents
LocalDate	date only (2026-10-17)
LocalTime	time only (14:30:05)
LocalDateTime	date + time
Instant	a moment on the time line (UTC)
DateTimeFormatter	format / parse text
TemporalAdjusters	shortcuts like "next Monday"

```java
LocalDate today = LocalDate.now();
Instant now = Instant.now();                        // java.time.Instant
DateTimeFormatter f = DateTimeFormatter.ofPattern("dd-MM-yyyy");
System.out.println(today.format(f));
LocalDate nm = today.with(TemporalAdjusters.next(DayOfWeek.MONDAY));
LocalDate last = today.with(TemporalAdjusters.lastDayOfMonth());
```

Pattern tip

Pattern letters: dd=day, MM=month, yyyy=year, HH=hour(24), mm=minute, ss=second.

(Capital M = month, small m = minute!)

4. Exception Handling

An ***exception*** is an unwanted event at ***run time*** that disturbs normal flow (divide by zero, file missing). Handling it prevents the program from crashing.

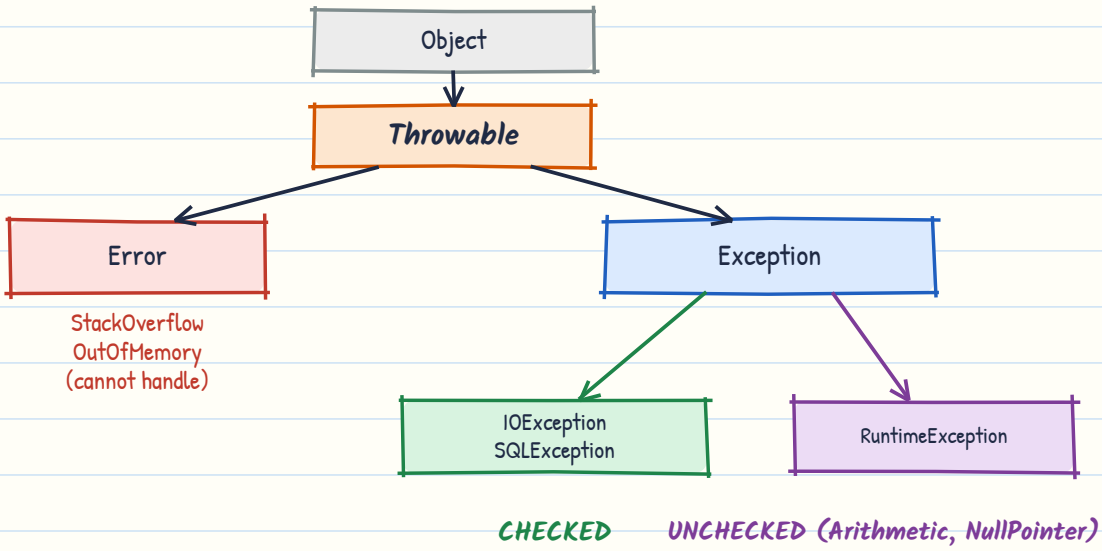


Fig 3: Exception class hierarchy

	Checked	Unchecked
Checked by	compiler (must handle/declare)	not checked at compile time
Examples	IOException, SQLException	ArithmeticException, NullPointerException
Parent	Exception	RuntimeException

try, catch, finally

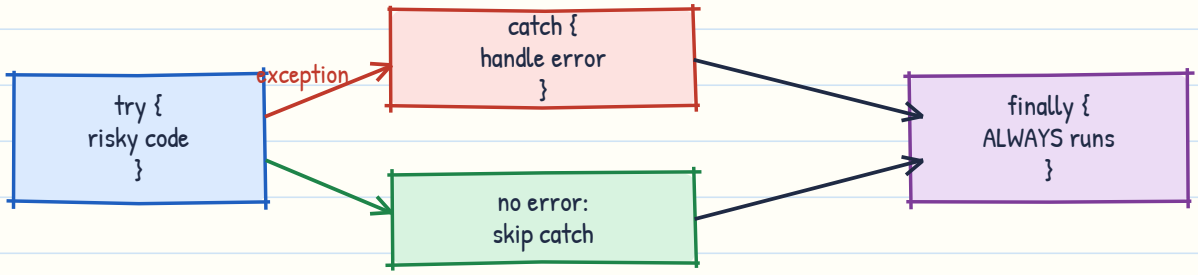


Fig 4: Flow of try-catch-finally

```
try {
    int r = 10 / 0;                    // throws ArithmeticException
    System.out.println(r);             // skipped
} catch (ArithmeticException e) {
    System.out.println("Error: " + e.getMessage());
} finally {
    System.out.println("cleanup always runs");
}
```

```
> OUTPUT
Error: / by zero
cleanup always runs
```

Multiple catch clauses

```
try {
    int[] a = new int[3];  a[5] = 1;
    String s = null;       s.length();
} catch (ArrayIndexOutOfBoundsException e) { System.out.println("bad index"); }
  catch (NullPointerException e)           { System.out.println("null!"); }
  catch (Exception e)                      { System.out.println("other"); }  // LAST
```

Rule

Order matters: put specific (child) exceptions first and general Exception last, otherwise compile error (unreachable catch).

throw and throws

Keyword	Meaning
throw	actually raises an exception object (inside method)
throws	declares in method header that it MAY raise it

```
static void check(int age) throws Exception {      // declares
    if (age < 18) throw new Exception("Not eligible");   // raises
    System.out.println("Eligible");
}
public static void main(String[] a) {
    try { check(15); } catch (Exception e) { System.out.println(e.getMessage()); }
}
```

```
> OUTPUT
Not eligible
```

Class Throwable & user-defined exceptions

Useful methods of *Throwable*: `getMessage()`, `printStackTrace()`, `toString()`.

Create your own exception by extending `Exception`.

```java
class MyEx extends Exception {
    MyEx(String msg) { super(msg); }
}
// throw new MyEx("custom problem");
```

5. Java I/O and Files

Java reads/writes data using ***streams*** – a flow of data between a program and a source/destination (keyboard, file, network).



Fig 5: Input stream & output stream

Standard I/O streams

Stream	Meaning	Object
System.in	input (keyboard)	InputStream
System.out	normal output (screen)	PrintStream
System.err	error output	PrintStream

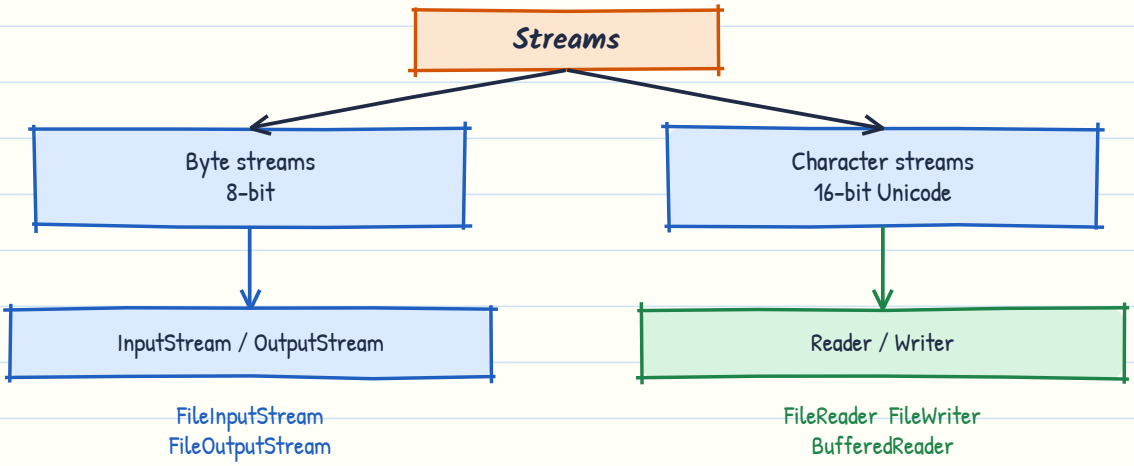


Fig 6: Types of streams

- ***Byte streams*** – images, audio, any binary data.
- ***Character streams*** – text files; handle Unicode & line reading.

Reading and writing files

```
import java.io.*;
public class FileDemo {
    public static void main(String[] a) throws IOException {
        FileWriter w = new FileWriter("data.txt");      // write
        w.write("Hello File");  w.close();
        FileReader r = new FileReader("data.txt");      // read
        int ch;
        while ((ch = r.read()) != -1) System.out.print((char) ch);
        r.close();
    }
}
```

```
> OUTPUT
Hello File
```

```
Scanner sc = new Scanner(new File("data.txt"));   // easy line reading
while (sc.hasNextLine()) System.out.println(sc.nextLine());
File f = new File("data.txt");
f.exists();  f.length();  f.getName();  f.delete();   // File class methods
```

Good practice

Always close streams (or use try-with-resources: try (FileReader r = new FileReader(..)) { }) so data is flushed and resources freed.

Quick Revision & Exam Questions

Topic	Memory line
package	first statement; folders; import to use
Wrapper	primitive <-> object, auto boxing/unboxing
Exception	Throwable -> Error / Exception -> Runtime
finally	always runs (cleanup)
throw / throws	raise / declare
Streams	byte (InputStream) vs character (Reader)

Likely exam questions

- What is a package? How do you create and import one?
- Explain wrapper classes with autoboxing and unboxing.
- Explain Math, Random and Formatter classes.
- Explain java.time classes and date formatting.
- Explain exception hierarchy; checked vs unchecked.
- Explain try, catch, finally, throw and throws with programs.
- Explain byte and character streams; write a file read/write program.